الافتراض والعزل في زمن الهزل

Lecture 3 — Virtualization Security

> **Traditional server architecture** : each physical server has its own CPU, RAM and storage, one OS installed directly on the hardware, and one service (web, mail, database).

Q/ What are the problems of the traditional architecture ?

1. Hardware **underutilization**. 2. Increased **costs**. 3. Higher **power** consumption. 4. More **maintenance**.

> **Virtualization** : running multiple operating systems on one physical machine by creating separate **virtual machines (VMs)**, each with its own virtual CPU, RAM, storage, OS and apps.

> **Hypervisor** : the software layer that **creates and runs VMs**, allocates physical CPU and RAM to each, and manages access to the hardware.

Type 1 hypervisor	Type 2 hypervisor
Runs directly on the hardware	Runs as an **application inside a host OS**
No general-purpose OS underneath	Layers: hardware → host OS → hypervisor → guest OSes
Controls hardware and allocates it to VMs	Host OS and hypervisor work together

Q/ What are the four benefits of virtualization ?

1. **Partitioning**: dividing one computer's resources among several VMs.
2. **Isolation**: each VM operates separately; a fault in one doesn't affect others (**fault isolation**) — but not absolute protection against hypervisor flaws or shared-hardware attacks.
3. **Encapsulation**: a VM and its configuration are stored as **files** → easy to copy, back up, move, restore.
4. **Hardware independence**: a VM isn't tied to one physical computer and can move to another compatible server without reinstalling.

Q/ What three mechanisms secure cloud virtualization ?

1. **Hardware isolation**: each VM has an isolated memory space.
2. **Hypervisor security**: the hypervisor separates VMs; **patches** fix weaknesses.
3. **Confidential computing**: AMD SEV / Intel SGX protect data **during processing**.

Confidential computing

Data state	Meaning	Traditional protection
At rest	Stored on disks, databases	Disk encryption
In transit	Moving over a network	TLS
In use	Being processed by the CPU, in memory	**Confidential computing**

> **Confidential computing** : hardware-based mechanisms that protect sensitive data **while it is processed**, isolating it from untrusted software (host OS, hypervisor).

> **Memory encryption** : data is **encrypted before it is written to RAM** and **decrypted when read**, by a hardware engine.

Q/ What are the steps of hardware memory encryption ?

1. Data is processed by the VM.
2. **Encryption before storage** by the memory-encryption engine.
3. **Encrypted storage** in RAM.
4. **Decryption during access** when the processor reads it.
5. **Transparent operation** — done in hardware; apps don't do it themselves.

> **AES (Advanced Encryption Standard)** : a **symmetric** algorithm (same key encrypts and decrypts) working on **128-bit blocks**.

AES key size	Rounds
AES-128	10
AES-192	12
AES-256	14

Q/ What are the AES round operations ?

1. **SubBytes** — replace each byte using a substitution table.
2. **ShiftRows** — shift rows to rearrange bytes.
3. **MixColumns** — combine bytes in each column (diffusion). **Not in the final round.**
4. **AddRoundKey** — XOR with the round key.

> **Encryption key** : the secret value the algorithm uses. **AES is the algorithm; the key is the value it uses.** Whoever gets the key can decrypt.

Q/ What does a hardware key mechanism do ?

1. **Key generation** — create a secure secret key.
2. **Key assignment** — tie it to a VM or protected environment.
3. **Key protection** — keep it inside a hardware boundary.
4. **Key use** — let the hardware use it without exposing it.
5. **Key isolation** — one VM can't use another's key.

Term	Meaning
Key generation	Creating a new secret key
Key derivation	Producing a specific key from a protected **root key** + other inputs (e.g. enclave identity)
Key management	Controlling assignment, use, protection, lifetime and access

AMD SEV

> **AMD SEV (Secure Encrypted Virtualization)** : protects the memory of an **entire VM** from the hypervisor and other software outside it.

Q/ What are the components of AMD SEV ?

1. **Guest VM** — its memory is encrypted in physical RAM.
2. **Hypervisor** — manages VMs but must **not** see plaintext VM memory.
3. **AMD Secure Processor (AMD-SP)** — initializes SEV, generates/manages VM-specific keys, keeps keys in hardware; the hypervisor **can't read the VM key**.
4. **Memory encryption engine** — in the memory-access path; encrypts/decrypts automatically.

Q/ How does SEV protect memory ?

1. AMD-SP initializes SEV. 2. Hypervisor creates a VM and requests SEV. 3. A VM-specific key is associated. 4. VM runs. 5. Writes are encrypted with the VM's key. 6. Reads are decrypted automatically. 7. If the hypervisor inspects the memory it sees only **ciphertext**.

> **VEK (VM Encryption Key)** : the secret key of an SEV-protected VM, used by the hardware to encrypt that VM's memory.

> **ASID (Address Space Identifier)** : identifies a VM's encryption context. **ASID is NOT an encryption key** and isn't secret.

Extension	Main protection
SEV	Encrypts VM memory (confidentiality from the hypervisor)
SEV-ES	+ protects **CPU register state** when the VM exits to the hypervisor
SEV-SNP	+ **memory integrity**: against malicious remapping and **replay** attacks

Encryption alone protects **confidentiality**, not integrity — that's why SEV-SNP matters.

Intel SGX

> **Intel SGX (Software Guard Extensions)** : protects **selected application code and data** inside an **enclave** (not the whole VM).

Q/ What are the components of Intel SGX ?

1. **Application outside the enclave** — ordinary, not automatically trusted.
2. **Enclave** — protected memory with selected code and secrets (e.g. crypto operations and keys).
3. **EPC (Enclave Page Cache)** — protected memory holding enclave pages.
4. **CPU and SGX key hierarchy** — hardware key material used to **derive** keys; not exposed to OS/hypervisor.

Q/ How does SGX protect data ?

1. Create an enclave. 2. Load protected code and data. 3. Establish enclave identity. 4. Enter with **ECALL**. 5. Process secrets inside. 6. Leave with **EEXIT** and return permitted results. 7. Hardware blocks the OS and hypervisor from reading enclave memory.

> **EGETKEY** : the instruction an enclave uses to ask the **CPU to derive a key** (the OS/hypervisor never receive the root secret).

> **Sealing** : encrypting enclave data so it can be **stored outside the enclave** and later recovered by an authorized enclave.

Q/ What are the steps of sealing ?

1. Hold a sensitive key. 2. Request a **sealing key** via EGETKEY. 3. Encrypt the data. 4. Store it outside (e.g. disk). 5. Later request the sealing key again and decrypt.

AMD SEV	Intel SGX
Protects the whole VM's memory	Protects **selected code/data** in an enclave
Keys managed by the AMD Secure Processor (VEK per VM)	Keys **derived by the CPU** (EGETKEY)
Hides the VM from the hypervisor	Hides the enclave from the OS and hypervisor
Extensions: SEV-ES, SEV-SNP	Sealing for storing secrets outside

Major virtualization threats

> **VM escape** : compromising a VM and then exploiting a hypervisor/virtualization flaw to reach the **host or other VMs**.

Q/ What are the steps of a VM escape ?

1. Compromise a VM (app/OS flaw in VM1).
2. Send crafted requests to a **virtual device** (network adapter, storage controller).
3. Trigger a hypervisor vulnerability → code runs **outside** the VM.
4. Escape to the host/hypervisor.
5. Attack other VMs or steal data.

> **Side-channel attack** : inferring secrets from **indirect information** (timing, cache behaviour) instead of reading data directly — possible because VMs share CPU cores and caches.

Q/ What are the steps of a cache side-channel between two VMs ?

1. Both VMs share one physical CPU. 2. VM1 processes secrets (keys, passwords). 3. Attacker in VM2 observes shared cache activity. 4. Measures timing/access patterns. 5. Infers VM1's secrets from repeated measurements.

Side-channel technique	Observes
Cache attacks	CPU cache access patterns and timing
Timing attacks	Execution time of sensitive operations
Speculative-execution attacks	Microarchitectural side effects of CPU speculation

> **SSRF (Server-Side Request Forgery)** : making a server send unauthorized requests to **internal** systems (cloud management interfaces, internal APIs, **metadata services**).

Q/ What are the steps of SSRF in the cloud ?

1. Find an app that fetches user-supplied URLs.
2. Supply a URL pointing to an **internal** service.
3. The server sends the request from **its own network**.
4. The internal service answers (inadequately restricted).
5. Abuse the metadata/credentials to reach more cloud resources.

Threat	Layer it targets
VM escape	The **isolation boundary** enforced by the hypervisor
Side-channel	**Shared hardware**
SSRF	A **vulnerable application** reaching internal cloud services
Insecure configuration	Settings across the stack

Summary of Advanced Cybersecurity L3: Virtualization Security. Built only from the lecture file; check the original slides for anything you're unsure about.